

1. Propositional and First-Order Logic

1. Classifying Statements (Tautology, Contradiction, or Contingent)

- a) $P \vee \neg P$
- This is a **tautology** because it is always true, regardless of the truth value of P . (Law of the excluded middle)
- b) $(P \vee Q) \wedge (\neg P \vee \neg Q)$
- This is a **contingent** statement because its truth value depends on the values of P and Q .
 - For example, if $P = \text{true}$ and $Q = \text{true}$, the expression evaluates to **false**, but for $P = \text{true}, Q = \text{false}$, it evaluates to **true**.
- c) $(P \rightarrow Q) \vee (Q \rightarrow P)$
- This is a **tautology** (principle of connexity).
 - Using a truth table, it is always **true** for any values of P and Q .
- d) $(P \wedge Q) \rightarrow P$
- This is a **tautology**, since if $P \wedge Q$ is true, then P must be true.
-

2. Truth Tables

(a) $(P \vee Q) \rightarrow R$

P	Q	R	$P \vee Q$	$(P \vee Q) \rightarrow R$
T	T	T	T	T
T	T	F	T	F
T	F	T	T	T
T	F	F	T	F
F	T	T	T	T
F	T	F	T	F
F	F	T	F	T
F	F	F	F	T

- The expression is not a tautology, since it is false in some cases.
-

3. Translating to Propositional Logic

- a) “If it is raining, then the ground is wet.”
- Let R be “It is raining” and W be “The ground is wet”.
 - $R \rightarrow W$.
- b) “Either Alice or Bob will win the competition, but not both.”

- Let A be “Alice wins” and B be “Bob wins”.
 - $(A \vee B) \wedge \neg(A \wedge B)$.
 - c) “If the alarm is not working, then either there is a power failure or the battery is dead.”
 - Let A be “The alarm is working”, P be “There is a power failure”, and B be “The battery is dead”.
 - $\neg A \rightarrow (P \vee B)$.
-

2. Formalization of Arguments

4. Checking Argument Validity

a) **Premises:**

- $P \rightarrow Q$

- $Q \rightarrow R$

- P

Conclusion: R

- By **Modus Ponens**, from $P \rightarrow Q$ and P , we infer Q .

- By **Modus Ponens** again, from $Q \rightarrow R$ and Q , we infer R .

- **Valid argument.**

b) **Premises:**

- $P \vee Q$

- $\neg P$

Conclusion: Q

- By **Disjunctive Syllogism**, if $P \vee Q$ is true and P is false, then Q must be true.

- **Valid argument.**

c) **Premises:**

- $(P \rightarrow Q) \wedge (Q \rightarrow R)$

- $\neg R$

Conclusion: $\neg P$

- Using **Contraposition**, $Q \rightarrow R$ is equivalent to $\neg R \rightarrow \neg Q$.

- Since $\neg R$ is given, we conclude $\neg Q$.

- By **Contraposition** again, $P \rightarrow Q$ implies $\neg Q \rightarrow \neg P$.

- Since $\neg Q$ is true, $\neg P$ follows.
 - **Valid argument.**
-

5. Natural Deduction Proofs

a) $P \vee (Q \wedge R) \vdash (P \vee Q) \wedge (P \vee R)$

1. Assume $P \vee (Q \wedge R)$.
2. Case 1: If P , then both $P \vee Q$ and $P \vee R$ hold.
3. Case 2: If $Q \wedge R$, then $P \vee Q$ (since Q is true) and $P \vee R$ (since R is true).
4. In both cases, $(P \vee Q) \wedge (P \vee R)$ follows.

b) $(P \rightarrow Q) \wedge (R \rightarrow S), P \vee R \vdash Q \vee S$

1. From $(P \rightarrow Q) \wedge (R \rightarrow S)$, we know both implications hold.
 2. If P , then Q follows.
 3. If R , then S follows.
 4. Since either P or R is true, then either Q or S must be true.
-

3. Applications in Mathematics and Computation

6. Set Theory

Given $U = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$, $A = \{2, 4, 6, 8\}$, $B = \{1, 2, 3, 4\}$:

- a) $A \cup B = \{1, 2, 3, 4, 6, 8\}$.
 - b) $A \cap B = \{2, 4\}$.
 - c) $A^c = \{1, 3, 5, 7, 9\}$.
 - d) $(A \cup B)^c = \{5, 7, 9\}$.
-

7. Boolean Algebra

- a) $(A \vee B) \wedge (\neg A \vee B)$

- Using **distribution**:
 - $(A \wedge \neg A) \vee (A \wedge B) \vee (B \wedge \neg A) \vee (B \wedge B)$
 - Since $A \wedge \neg A = 0$ and $B \wedge B = B$:
 - Result: $(A \wedge B) \vee B = B$.
 - b) $(A \wedge B) \vee (\neg A \wedge B)$
 - Factor out B :
 - $B \wedge (A \vee \neg A)$.
 - Since $A \vee \neg A = 1$:
 - Result: B .
 - c) $A \vee (A \wedge B)$
 - Using **Absorption**: $A \vee (A \wedge B) = A$.
-

8. Logic Programming (Prolog)

a) Recursive rule:

```
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

b) Query:

```
?- ancestor(alice, david).
```

Result: **true**.